CÁTEDRA DE PROGRAMACIÓN ORIENTADA A OBJETOS 3 (POO3)

INFORME FINAL DE PROYECTO
SISTEMA "VetTurnos"

Plataforma Web Integral para la Gestión, Reserva y Control de Turnos en Clínicas Veterinarias de Animales Chicos con Modelado de Ciclo de Vida y Notificaciones

Materia:	Programación Orientada a Objetos 3 (POO3)
Proyecto:	VetTurnos — Sistema de Gestión Veterinaria
Stack Tecnológico:	Python 3 + Flask + Flask-SQLAlchemy (ORM) + SQLite + Jinja2
Metodología & Gestión:	Agile / Scrum con Atlassian Jira (Backlog, CU-01 a CU-16, HU-01 a HU-13)
Repositorio Oficial:	https://github.com/JonathanCondori9/POO3---Proyecto-Final

1. Introducción y Contexto del Negocio

El presente informe documenta el diseño, modelado orientado a objetos e implementación de la plataforma web VetTurnos, desarrollada en el marco de la asignatura Programación Orientada a Objetos 3 (POO3). La solución aborda la problemática crítica de gestión y programación de turnos en clínicas veterinarias especializadas en animales chicos (caninos y felinos).

A diferencia de un turnero genérico o comercial estándar, una clínica veterinaria posee particularidades operativas no lineales que requieren un modelado de software riguroso:

- **Disparidad de Duraciones y Requerimientos de Infraestructura:** Prácticas de rutina como consultas clínicas generales, desparasitaciones o vacunas tienen duraciones breves (20 a 30 minutos) y están habilitadas de lunes a sábados por cualquier veterinario generalista.
- **Prácticas Quirúrgicas y Diagnóstico por Imágenes:** Procedimientos quirúrgicos complejos como castraciones y esterilizaciones demandan bloques de 90 minutos y solo pueden ejecutarse en días fijos de quirófano con anestesia inhalatoria (martes y jueves). Por su parte, las ecografías requieren equipamiento específico y presencia del ecografista de turno (miércoles y viernes).
- **Problemática del Ausentismo y Liberación Dinámica de Cupos:** En este rubro, las inasistencias sin previo aviso perjudican la operatividad médica. Se requirió un ciclo de vida con estados formales para auditar ausentismos y liberar cupos automáticamente ante cancelaciones anticipadas.

> **MISIÓN DEL SISTEMA VETTURNOS**
> Objetivo Central del Proyecto: Diseñar un sistema web robusto bajo el paradigma orientado a objetos (POO) que automatice la agenda médica veterinaria, desacople la asignación de profesionales por especialidad, gestione el ciclo de vida del turno con máquinas de estado e implemente un Centro de Notificaciones reactivo para clientes y personal administrativo.

2. Metodología Ágil y Gestión de Requerimientos (Jira)

Para garantizar una trazabilidad profesional en el desarrollo, el equipo adoptó la metodología Scrum apoyada en la plataforma Atlassian Jira. Se relevaron 16 Casos de Uso formales y se desagregaron en 13 Historias de Usuario (HU) con Criterios de Aceptación verificables redactados en formato Gherkin (Dado que / Cuando / Entonces).

2.1 Matriz de Casos de Uso del Sistema

Identificador	Nombre del Caso de Uso	Alcance / Actor Principal
CU-01 / CU-02	Crear cuenta & Iniciar Sesión	Autenticación segura con hashing de passwords (Cliente/Admin).
CU-03	Modificar perfil de usuario	Actualización de datos personales y teléfono de contacto.
CU-04	Gestionar perfil mascota	Alta y edición de perros/gatos con peso, género, edad y antecedentes.
CU-05	Visualizar horarios y disponibilidad	Motor de consulta de slots hábiles libres en franja

		de 09:00 a 18:00 hs.
CU-06	Reservar turno general	Agendamiento de consultas de rutina (30 min) en estado PENDIENTE.
CU-07	Reservar turno particular	Reserva de cirugías/castraciones y ecografías con días restringidos.
CU-08	Modificar / Reprogramar turno	Cambio de fecha y horario liberando automáticamente el cupo previo.
CU-09	Cancelar turno	Transición a estado CANCELADO; liberación inmediata del cupo para otros clientes.
CU-10	Ver historial de turnos	Trazabilidad cronológica de citas pasadas y activas con estado visible.
CU-11	Ver notificaciones	Bandeja de avisos por confirmación, cancelación o reasignación.
CU-12	Gestión de agenda y Control de Asistencia	Supervisión diaria: marcar turnos como ATENDIDO o AUSENTE.
CU-13	Asignar turno de control	Generación directa de consultas de seguimiento clínico posterior.
CU-14	Reasignación operativa de turnos	Movimiento de citas por imprevistos con despacho de alerta al cliente.
CU-15	Consultar datos cliente / mascota	Buscador y ficha completa con historial médico y tasa de presentismo.
CU-16	Gestionar staff veterinario	ABM de veterinarios: matrícula profesional, especialidades y estado.

2.2 Caso de Estudio de Requerimientos: HU-09 Centro de Notificaciones (Ticket Jira KAN-17)

Como ejemplo de aplicación rigurosa de Criterios de Aceptación, la Historia de Usuario HU-09 fue estructurada en cuatro escenarios exhaustivos que guiaron la implementación final del sistema:

- **Escenario 1 (Confirmación de Turno):** Al confirmar con éxito una reserva (cliente o admin), el sistema persiste el registro en la tabla NOTIFICACIONES vinculado al usuario y actualiza el contador de alertas en el navbar.
- **Escenario 2 (Reasignación Operativa):** Cuando un administrador reprograma fecha, hora o médico de una cita pendiente, se despacha una notificación prioritaria clasificada como 'REASIGNACION' al perfil del cliente afectado.
- **Escenario 3 (Marcar como Leída):** Al ingresar al Centro de Notificaciones, el usuario puede marcar alertas individuales o presionar 'Marcar todas como leídas', actualizando el flag booleano 'leida=True' y reduciendo el badge numérico.
- **Escenario 4 (Bandeja Vacía / Historial):** Cuando la bandeja no posee mensajes pendientes, se despliega un mensaje informativo claro: 'No tienes notificaciones nuevas en este momento', manteniendo visible el historial pasado.

3. Arquitectura del Software y Patrones de Diseño

El sistema adopta una arquitectura multicapa estructurada bajo el patrón Modelo-Vista-Controlador / Modelo-Vista-Template (MVC / MVT) con Renderizado del Lado del Servidor (Server-Side Rendering - SSR), garantizando rendimiento, simplicidad y robustez sin dependencias de frameworks externos pesados de frontend.

3.1 Patrones de Diseño de Software Aplicados

- **Application Factory Pattern:** Implementado en app.py a través de la función create_app(config_class). Permite instanciar la aplicación de forma dinámica, configurar extensiones como Flask-SQLAlchemy y crear el esquema de base de datos de manera aislada y comprobable.
- **Modular Routing con Blueprints:** En lugar de centralizar todas las rutas en un único archivo monolítico, el backend se desacopló en 6 submódulos autónomos mediante Flask Blueprints (main_bp, auth_bp, mascota_bp, turno_bp, admin_bp, notificacion_bp). Cada uno administra su propio prefijo de URL y lógica de controlador.
- **Decoradores de Control de Acceso (Decorator Pattern):** Se implementaron los decoradores personalizados @login_required y @admin_required en routes/auth_utils.py. Interceptan peticiones HTTP, validan la existencia de la sesión y auditan que el usuario autenticado siga activo en la base de datos, impidiendo accesos no autorizados o inconsistencias de sesión huérfana.
- **Context Processor Global (Inyección de Dependencias en Vistas):** Se utilizó @app.context_processor para inyectar automáticamente en todas las plantillas HTML el conteo reactivo de alertas pendientes (notificaciones_no_leidas_count), alimentando la campana del navbar sin duplicar código en los controladores.
- **Endpoints JSON Internos (Arquitectura Híbrida Web/API):** La selección interactiva de horarios disponibles en la pantalla de reserva no recarga la página: consume el endpoint JSON interno GET /turnos/api/disponibilidad vía Fetch API vanilla (JavaScript nativo puro).

4. Diseño y Modelado Orientado a Objetos (POO)

El núcleo de VetTurnos reside en el modelado de clases de dominio utilizando el ORM Flask-SQLAlchemy. A través de este enfoque, se erradicó por completo la escritura de sentencias SQL crudas, modelando las entidades como objetos reales de Python dotados de estado, comportamiento y métodos de negocio.

4.1 Entidades del Dominio POO

- **Usuario (models/usuario.py)** Modela a Clientes y Administradores. Aplica encapsulamiento seguro mediante set_password() y check_password() utilizando el algoritmo criptográfico pbkdf2:sha256 de Werkzeug. Posee la propiedad computada es_admin y relaciones 1:N con Mascotas, Turnos y Notificaciones.
- **Mascota (models/mascota.py)** Representa a los pacientes caninos y felinos. Incluye atributos clínicos fundamentales incorporados en la última fase: peso (kg), genero (Macho/Hembra), edad, raza y observaciones. Su relación con Usuario implementa borrado en cascada (cascade='all, delete-orphan').
- **Veterinario (models/veterinario.py)** Modela al personal médico profesional. Contiene matricula profesional habilitante (con restricción de unicidad unique=True), especialidad médica, datos de contacto y un flag activo para altas/bajas lógicas.

- **TipoTurno (models/tipo_turno.py)** Parametriza las prácticas médicas de la clínica: categoria (General, Particular, Control), practica (Consulta, Vacunación, Castración, Ecografía, Análisis), duracion_minutos (20, 30, 45, 90) y dias_habilitados.
- **Turno (models/turno.py)** Entidad nuclear de la clínica. Modela la cita médica interconectando Usuario, Mascota, Veterinario y TipoTurno mediante claves foráneas y relaciones bidireccionales back_populates. Encapsula toda la máquina de estados del ciclo de vida.
- **Notificacion (models/notificacion.py)** Mensajería interna del sistema. Clasifica las alertas por tipo (CONFIRMACION, REASIGNACION, CANCELACION, INFO), fecha de envío y estado de lectura (leida=False/True).

4.2 Máquina de Estados del Turno (State Pattern / Ciclo de Vida)

Uno de los puntos más valorados académicamente en POO3 es el modelado del Ciclo de Vida del Turno. Un turno no es un registro estático, sino un objeto que atraviesa transiciones controladas por métodos propios de la clase Turno:

Estado	Método POO Ejecutado	Impacto en la Regla de Negocio y en la Agenda
PENDIENTE	reasignar(nueva_fecha, vet)	Bloquea el cupo horario en la agenda. La propiedad turno.bloquea_agenda retorna True. Ningún otro cliente puede seleccionar ese slot.
ATENDIDO	marcar_atendido(obs)	Transiciona cuando la mascota asistió y el veterinario completó la atención. Permite asentar observaciones médicas en el historial.
AUSENTE	marcar_ausente()	Se ejecuta si llegó la hora y el paciente no se presentó. Permite al administrador computar métricas de inasistencia.
CANCELADO	cancelar()	Anulación solicitada por el cliente o admin. Transiciona el estado a CANCELADO y libera de inmediato el horario para el resto del público.

5. Módulos Funcionales Implementados

A continuación se detallan los módulos de software construidos que componen la versión final operativa del sistema:

5.1 Módulo 1: Autenticación, Seguridad y Perfil (routes/auth_routes.py)

- **Registro de Clientes (CU-01):** Validación exhaustiva de formato de correo, contraseña mínima de 6 caracteres y coincidencia de confirmación. Almacenamiento seguro mediante hashes PBKDF2.
- **Inicio de Sesión y Roles (CU-02):** Identificación unificada de Clientes y Administradores. Redirección condicional según rol (los administradores al Dashboard /admin/ y los clientes a sus mascotas /mascotas/). Se removieron las credenciales demo en pantalla para una presentación limpia de producción.
- **Gestión de Perfil (CU-03):** Vista protegida /auth/perfil (con alias directo /perfil) para actualizar nombre, apellido, teléfono y cambio voluntario de clave. Manejo de sesiones huérfanas ante reinicios de base de datos.

5.2 Módulo 2: Gestión de Pacientes Caninos y Felinos (routes/mascota_routes.py)

- **Alta y Edición de Mascotas (CU-04):** Formulario con validación estricta de especie (Perro/Gato/Otro animal chico), nombre, raza, edad y los dos campos médicos solicitados: Peso en kilogramos (ej: '28 kg') y Género ('Macho ♂' / 'Hembra ♀').
- **Aislamiento y Seguridad Multiusuario:** Cada petición valida que mascota.usuario_id == session['usuario_id']. Un cliente no puede ver, modificar ni solicitar turnos para animales de otros dueños.
- **Visualización en 'Mis Mascotas':** Cuadrícula dinámica con tarjetas visuales, distintivo de especie mediante emojis temáticos (🐶 / 🐱) y acceso directo 'Solicitar Turno' preseleccionando el paciente.

5.3 Módulo 3: Motor de Agenda y Reserva de Turnos (routes/agenda_service.py y turno_routes.py)

- **Algoritmo de Disponibilidad Horaria:** El servicio calcula franjas horarias libres entre las 09:00 y las 18:00 hs considerando la duración exacta de la práctica (ej. intervalos de 30 min para consulta, 90 min para castración). Bloquea domingos y valida los días habilitados de quirófano.
- **Asignación Médica por Especialidad:** Al reservar, el sistema asigna automáticamente un profesional adecuado: para cirugías/castraciones busca veterinarios con especialidad en 'Cirugía'; para ecografías, busca 'Ecografía'; en consultas generales, asigna al clínico activo disponible.
- **Reserva Asistida (CU-06 / CU-07):** Selector de horarios interactivo en el cliente mediante botones 'pills' que consultan la disponibilidad en tiempo real.
- **Modificación y Cancelación (CU-08 / CU-09):** El cliente puede modificar día/hora o anular su cita. En ambos casos, el slot anterior queda liberado inmediatamente en la agenda general.

5.4 Módulo 4: Panel de Control Administrativo (routes/admin_routes.py)

- **Dashboard con Estadísticas Diarias (CU-11 / CU-14):** Métricas clave en tiempo real: turnos pendientes, atendidos y ausentes de la fecha, total de clientes, mascotas registradas y volumen del mes.
- **Agenda Diaria y Vista Global de Pendientes (CU-12):** Permite filtrar turnos por fecha y profesional, además del botón destacado '🕐 Ver Todos los Pendientes' para auditar la totalidad de citas pendientes sin importar el día.

- **Control de Asistencia en Vivo (CU-12):** Botones directos '☑ Atendido' y '⚠ Ausente' con confirmación previa y registro de observaciones clínicas.
- **Reasignación Operativa de Citas (CU-14):** En /admin/turno/<id>/reprogramar el personal administrativo puede cambiar fecha, hora y veterinario de una cita, despachando automáticamente una notificación de reasignación al cliente.
- **ABM de Veterinarios (CU-16) y Directorio de Clientes (CU-15):** Altas, modificaciones y activación/desactivación de médicos con validación de matrícula profesional única.

5.5 Módulo 5: Centro de Notificaciones (routes/notificacion_routes.py)

- **Campana Interactiva en Navbar:** Ubicada en la barra de navegación al lado de 'Cerrar Sesión', con burbuja flotante del contador de alertas no leídas.
- **Generación Automática de Notificaciones:** Confirmaciones de reserva, reasignaciones de turnos y avisos de inasistencia.
- **Bandeja de Entrada Integral (HU-09):** Marcación de notificaciones individuales, acción global '✓ Marcar todas como leídas' y visualización de bandeja limpia con historial previo.

6. Diseño de Interfaz de Usuario (UI/UX)

La interfaz visual de VetTurnos fue concebida con un enfoque profesional, empático y orientado a la accesibilidad:

- **Paleta de Colores Institucional:** Violeta institucional (#855ad0) como color primario, asociado a la innovación y calidez médica, complementado con blancos puros (#ffffff) y tonos neutros claros (#f9f8fc).
- **Composición Visual y Fondo Temático:** Fondo visual con fotografía de caninos y felinos optimizada localmente en static/images/fondo_mascotas.jpg, tratada con un filtro de overlay translúcido (rgba(249, 248, 252, 0.85)) que mantiene legibles los textos y tarjetas sin generar fatiga visual.
- **Badges de Estado Semánticos:** Cada estado de turno se distingue mediante un código cromático intuitivo: Violeta para PENDIENTE, Verde (#38a169) para ATENDIDO, Amarillo/Naranja (#dd6b20) para AUSENTE y Gris neutro para CANCELADO.
- **Formularios Guiados y Prevención de Errores:** Los formularios presentan validaciones HTML5 nativas, etiquetas de obligatoriedad claras (*), mensajes de ayuda y cuadros de diálogo de confirmación en acciones críticas.

7. Stack Tecnológico y Control de Versiones

Componente	Tecnología / Librería	Propósito en el Proyecto
Lenguaje Base	Python 3.13	Lenguaje principal del backend orientado a objetos.
Framework Web	Flask 3.x	Ruteo modular con Blueprints y arquitectura MVC.
Persistencia / ORM	Flask-SQLAlchemy (SQLAlchemy 2.x)	Mapeo objeto-relacional, encapsulamiento y relaciones POO.
Base de Datos	SQLite 3	Base de datos relacional integrada en instance/veterinaria.db.
Seguridad / Criptografía	Werkzeug Security	Hashing PBKDF2:SHA256 para almacenamiento seguro de credenciales.
Motor de Plantillas	Jinja2	Renderizado del lado del servidor (SSR) con herencia de plantillas.
Frontend & Estilos	HTML5 Semántico + CSS3 Puro	Diseño responsivo sin dependencias de librerías JS invasivas.

7.1 Trazabilidad en GitHub

El código fuente completo, el historial de commits paso a paso y la documentación técnica se encuentran alojados y sincronizados en el repositorio público de GitHub:

Enlace del Repositorio: https://github.com/JonathanCondori9/POO3---Proyecto-Final

8. Conclusiones y Logros Académicos

El desarrollo del proyecto VetTurnos permitió consolidar de forma práctica e interdisciplinaria los contenidos troncales de la materia Programación Orientada a Objetos 3:

- **Dominio del Paradigma POO:** Se comprobó la superioridad de trabajar con clases de dominio que encapsulan comportamiento frente a scripts procedurales con SQL disperso.
- **Persistencia Robusta mediante ORM:** La adopción de Flask-SQLAlchemy evidenció cómo las relaciones de agregación y composición del modelo conceptual se reflejan naturalmente en objetos navegables de software.
- **Control de Procesos de Negocio Complejos:** El modelado explícito de estados (Pendiente, Atendido, Ausente, Cancelado) resolvió de forma elegante la concurrencia en la reserva de turnos y la liberación automática de cupos.
- **Madurez en Metodologías Ágiles:** La formulación de Historias de Usuario con Criterios de Aceptación verificables en Jira garantizó que cada requerimiento fuera probado e integrado sin desvíos funcionales.

> **DICTAMEN FINAL**
> VetTurnos representa una solución de software completa, funcional y alineada a las mejores prácticas de la industria: código limpio, arquitectura desacoplada, seguridad en credenciales, pruebas de integración superadas y documentación formal para el usuario y el evaluador académico.